

INDEX

Name Adeebuddin Muhammad Std 4 Sec A

Roll No. BF16 Subject OS Lab School/College BMSCE

School/College Tel. No. _____ Parents Tel. No. _____

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
i	2/3	2 nd smallest / Largest element	8	
ii	2/3	Sum of left diagonal		
iii	2/3	Sum of rows & columns		
iv	2/3	Duplicate elements.		
v	2/3	Delete Element at position		
1	2/3	Turn around time and Wait time for FCFS and SJF	10	8/3
2.	10/3	TAT and WT for Priority (PE & NPE)	10	8/3 10/3
3.	17/3	Round Robin	9	8/3 17/3
4.	24/3	Multilevel queue.	9	8/3
5.	31/3	Realtime CPU scheduling.	9	8/3 9/3
6	21/4	Producer - Consumer	10	8/3 22/4
-	21/4	Dining - Philosopher.		
7	28/4	Banker's Algorithm	10	8/3 28/4
8	5/5	Deadlock Detection	8	8/3 5/5

Q: Write a program to find second smallest/largest element of an array

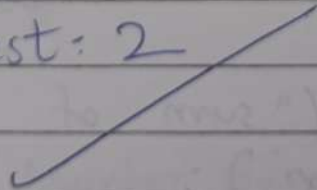
```
#include <stdio.h>

int main () {
    int arr[] = {5, 4, 3, 2, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    printf("2nd largest: %d", arr[n-2]);
    printf("2nd smallest: %d", arr[1]);
    return 0;
}
```

Output:

2nd largest : 4

2nd smallest : 2



Q. Write a program to find the sum of rows and columns of a matrix

```
#include <stdio.h>
int main() {
    int arr[3][3] = {1, 1, 2, 2, 2, 3, 3, 3};
    int sum;
    for (int i = 0; i < 3; i++) {
        sum = 0;
        for (int j = 0; j < 3; j++) {
            sum += arr[i][j];
        }
        printf("sum of row %d is %d", i, sum);
    }
    printf("\n");
    for (int i = 0; i < 3; i++) {
        sum = 0;
        for (int j = 0; j < 3; j++) {
            sum += arr[j][i];
        }
        printf("sum of col %d is %d", i, sum);
    }
    return 0;
}
```

Output:

Sum of row 0 is 6
 sum of row 1 is 6
 sum of row 2 is 9
 sum of col 0 is 6
 sum of col 1 is 6
 sum of col 2 is 9

Q. Write a program to count duplicate elements in an array.

```
#include <stdio.h>
int main() {
    int a[100], n, i, j, count = 0;
    printf("Enter size: "); scanf("%d", &n);
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                count++;
                break;
            }
        }
    }
    printf("Number of duplicates = %d", count);
}
```

Output:

Enter size: 10
 1 2 2 3 4 5 6 7 8 8

Number of duplicates: 2

Q. Write a program to delete an element at desired position.

```
#include <stdio.h>
int main() {
    int arr[] = {0, 1, 2, 3};
    int del_index = 1;
    int n = sizeof(arr) / sizeof(arr[0]);
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
    }
    printf("\n");
}
```

Q. Write a program to find the sum of left diagonals of a matrix.

```
#include <stdio.h>
int main() {
    int a[10][10], i, n, j, sum = 0;
    printf("Enter order of matrix: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    for (int i = 0; i < n; i++) {
        sum += a[i][i];
    }
    printf("Sum of left diagonal: %d", sum);
    return 0;
}
```

Output:

Enter order of matrix: 3
Elements: 1 3 5 6 7 8 9 2 4

Sum of left diagonal: 12

Q. Write a program to delete element at desired position:

```
#include <stdio.h>
int main()
{
    int a[200], n, pos, i;
    printf("Enter size: ");
    scanf("%d", &n);
    for (i=0; i<n; i++)
        scanf("%d", &a[i]);
    printf("Enter position to delete: ");
    scanf("%d", &pos);

    for (pos-1; i<n-1; i++)
        a[i] = a[i+1];

    n--;
    printf("Array after deletion: ");
    for (i=0; i<n; i++)
        printf("%d", &a[i]);
}
```

Output:

Enter size: 6
1 2 3 4 5 6
Enter position to delete: 4
Array after deletion: 1 2 3 5 6

Handwritten signature

Write program to find Turnaround time and waiting time for FCFS and SJF FCFS

```
#include <stdio.h>

struct Processes
{
    int ID, AT, BT, CT, TAT, WT, RT;
};

int main()
{
    int n;
    printf("Enter no. of processes: ");
    scanf("%d", &n);

    struct Processes p[n];
    for (int i=0; i<n; i++)
    {
        p[i].ID = i+1;
        printf("Arrival time of P%d: ", p[i].ID);
        scanf("%d", &p[i].AT);

        printf("Burst time of P%d: ", p[i].ID);
        scanf("%d", &p[i].BT);
    }

    for (int i=0; i<n; i++)
    {
        for (int j=0; j<n; j++)
            if (p[j].AT > p[i].AT)
            {
                struct Processes temp = p[j];
                p[j] = p[i];
                p[i+1] = temp;
            }
    }
}
```

Outputs

Enter number of processes: 2

Arrival Time of P1: 0

Burst Time of P1: 3

Arrival Time of P2: 1

Burst Time of P2: 2

Process	AT	BT	CT	TAT	WT	RT
P1	0	3	3	3	0	0
P2	1	2	5	4	2	2

Average TAT: 3.50

Average WT: 1.00

Average RT: 1.00

if timePassed < 0, sumTAT = 0, sumWT = 0;

for (int i = 0; i < n; i++) {

if (timePassed < p[i].AT) {

timePassed = p[i].AT;

}

p[i].CT = timePassed + p[i].BT;

p[i].TAT = p[i].CT - p[i].AT;

p[i].WT = p[i].TAT - p[i].BT;

p[i].RT = p[i].WT;

}

timePassed += p[i].BT;

sumTAT += p[i].TAT;

sumWT += p[i].WT;

sumRT += p[i].RT;

}

printf ("n Process \t TAT \t BT \t CT \t

TAT \t WT \t RT \n");

for (int i = 0; i < n; i++) {

printf ("p: %d \t %d \t %d \t %d \t %d \t %d \t %d \n", p[i].AT,

p[i].BT, p[i].CT, p[i].TAT, p[i].WT,

p[i].RT);

}

printf ("n Average TAT: %.2f \n", (float)

sumTAT / n);

printf ("n Average WT: %.2f \n", (float) sumWT /

n);

printf ("n Average RT: %.2f \n", (float) sumRT /

n);

SSE (non preemptive)

#include <stdio.h>

struct process {

int id, at, bt, ct, tot, wt;

};

int main() {

int n;

printf("Enter number of processes: ");

scanf("%d", &n);

struct process p[n];

for (int i = 0; i < n; i++) {

p[i].id = i + 1;

printf("Arrival time of %d:",

p[i].id);

scanf("%d", &p[i].at);

printf("Burst time of %d:",

p[i].id);

scanf("%d", &p[i].bt);

int completed = 0, time = 0;

while (completed != n) {

int idx = -1;

for (int i = 0; i < n; i++) {

if (p[i].completed == 0 && p[i].wt == 0) {

idx = i;

break;

if (idx == -1) {

printf("No process found to execute");

return 1;

int mbrt = 169;

for (int i = 0; i < n; i++) {

if (p[i].at < time || (p[i].at == time && p[i].bt < p[idx].bt)) {

idx = i;

if (p[i].bt < mbrt) {

mbrt = p[i].bt;

idx = i;

}

}

if (idx == -1) {

printf("No process found to execute");

return 1;

p[idx].wt = time - p[idx].at;

p[idx].ct = p[idx].bt + p[idx].wt;

sumTAT += p[idx].ct;

sumWT += p[idx].wt;

time = p[idx].bt;

p[idx].completed = 1;

completed++;

if (idx == -1) {

printf("No process found to execute");

return 1;

int mbrt = 169;

for (int i = 0; i < n; i++) {

printf("Process %d: at %d, bt %d, ct %d, wt %d",

p[i].id, p[i].at, p[i].bt, p[i].ct, p[i].wt);

printf("\n");

if (p[i].bt < mbrt) {

mbrt = p[i].bt;

idx = i;

```
printf("In Average TAT: %.2f\n",
(float) sumTAT/n);
printf("In Average WT: %.2f\n",
(float) sumWT/n);
return 0;
```

3

Output:

Enter number of processes: 2
 Arrival time of p1: 0
 Burst Time of p1: 3
 Arrival Time of p2: 1
 Burst Time of p2: 2

Process	AT	BT	CT	TAT	WT
p1	0	3	3	3	0
p2	1	2	5	4	2

Average TAT: 3.50
 Average WT: 1.00

SJF (Pre-emptive)

```
#include <stdio.h>
struct pprocess {
    int id, at, bt, ct, tot, wt, tt, rem;
```

```
int main() {
    int n;
    printf("Enter no. of process: ");
    scanf("%d", &n);
```

```
    struct Pprocess [n];
    for (int i=0; i<n; i++) {
        p[i].id = i+1;
        printf("Arrival Time of P%d: ", i+1);
        scanf("%d", &p[i].at);
        printf("Burst Time of P%d: ", i+1);
        scanf("%d", &p[i].bt);
        p[i].rem = p[i].bt; p[i].ct = -1;
```

3

```
    int completed=0, time=0, sumTAT=0,
    sumWT=0;
    while (completed != n) {
        int idx = -1, minRem = 1e9;
```

```
        for (int i=0; i<n; i++) {
            if (p[i].at <= time && p[i].rem > 0) {
                if (p[i].rem < minRem) {
                    minRem = p[i].rem;
                    idx = i;
```

3

```

if (idx == -1) {
    if (p[idx].rt == -1) {
        p[idx].rt = time - p[idx].at;
        sumRT += p[idx].rt;
    }
    p[idx].rt = time - p[idx].at;
    p[idx].rem -= 1; time++;
    if (p[idx].rem == 0) {
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].rt;

        sumTAT += p[idx].tat;
        sumWT += p[idx].wt;
        completed++;
    }
    else { time++; }

    printf("Avg TAT: %.2f\n", (float) sumTAT/n);
    printf("Avg WT: %.2f\n", (float) sumWT/n);
    printf("Avg RT: %.2f\n", (float) sumRT/n);
    return 0;
}

```

Output:

Enter no. of process: 3
 Arrival Time of P1: 0
 Burst Time of P1: 5
 Arrival Time of P2: 1
 Burst Time of P2: 3
 Arrival Time of P3: 2
 Burst Time of P3: 1

Avg TAT: 4.67
 Avg WT: 1.67

Process	AT	BT	CT	TAT	WT
P1	0	5	9	9	4
P2	1	3	5	4	1
P3	2	1	3	1	0

Comparison:

Algorithm	Avg WT	Avg TAT
FCFS	3.33	6.33
SJF (NPE)	2.67	5.67
SJF (PE)	1.67	4.67

SJF - is optimal

Signature

Week-2

Write a C program to simulate the following CPU scheduling algorithm to find TAT & WT for Priority (pre-emptive and non-pre-emptive)

```
#include <stdio.h>
```

```
struct Process {
    int pid, bt, at, pr, ct, tat, wt, rtfinished;
};
```

```
void priority_non_preemptive(struct Process p[], int n) {
```

```
    int time = 0, completed = 0;
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!p[i].finished &&
```

```
                p[i].at <= time) {
```

```
                if (p[i].pr < highest) {
```

```
                    highest = p[i].pr;
```

```
                    idx = i;
```

```
                }
```

```
            }
```

```
        }
```

```
        if (idx == -1) {
```

```
            time++;
```

```
            continue;
```

```
        }
```

```
        time += p[idx].bt;
```

```
        p[idx].ct = time;
```

```
        p[idx].tat = p[idx].ct - p[idx].at;
```

```
        p[idx].wt = p[idx].tat - p[idx].bt;
```

```
        p[idx].finished = 1;
```

```
        completed++;
```

```
    }
```

```
printf("\n-- Non-preemptive Priority scheduling ---\n");
```

```
printf("PID\tBT\tAT\tCT\tTAT\tWT\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d\t%d\t%d\t%d\t%d\t%d\t", p[i].pid,
```

```
        p[i].bt, p[i].at, p[i].ct, p[i].tat, p[i].wt);
```

```
    printf("\n");
```

```
    p[i].tat = p[i].ct - p[i].at;
```

```
    p[i].wt = p[i].tat - p[i].bt;
```

```
}
```

```
void priority_preemptive(struct Process p[], int n) {
```

```
    int time = 0, completed = 0;
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest = 9999;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!p[i].finished &&
```

```
                p[i].at <= time) {
```

```
                if (p[i].pr < highest) {
```

```
                    highest = p[i].pr;
```

```
                    idx = i;
```

```
            }
```

```
        }
```

```
    }
```

```

if (idx == -1) {
    time++;
    continue;
}
p[idx].rt--;
time++;
if (p[idx].rt == 0) {
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].finished = 1;
    completed++;
}
}

```

printf("\n---Preemptive Priority Scheduling---\n");

```

printf("PID\tAT\tBT\tRT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].rt, p[i].ct, p[i].tat, p[i].wt);
}
}

```

```

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
}

```

```

struct process p[n];
for (int i = 0; i < n; i++) {
    p[i].pid = i+1;
    printf("Enter AT, BT, Priority for P%d: ", i+1);
    scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
    p[i].rt = p[i].bt;
    p[i].finished = 0;
}
}

```

```

struct process p1[n], p2[n];
for (int i = 0; i < n; i++) {
    p1[i] = p[i];
    p2[i] = p[i];
}
}

```

```

priority = non-preemptive(p1, n);
priority = preemptive(p2, n);
return 0;
}

```

Output:

Enter number of processes: 3

Enter AT, BT, Priority for P1: 0 7 2

Enter AT, BT, Priority for P2: 2 3 1

Enter AT, BT, Priority for P3: 3 2 3

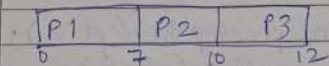
--- Non-preemptive Priority Scheduling ---

PID	AT	BT	PR	CT	TAT	WT
1	0	7	2	7	7	0
2	2	3	1	10	8	5
3	3	2	3	12	9	7

$$AVG-TAT = 8.00$$

$$AVG-WT = 4.00$$

--- Gantt Chart ---



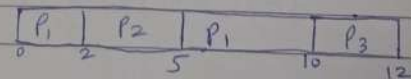
--- Preemptive Priority Scheduling ---

PID	AT	BT	PR	CT	TAT	WT
1	0	7	2	10	10	3
2	2	3	1	5	3	0
3	3	2	3	12	9	7

$$AVG-TAT = 7.33$$

$$AVG-WT = 3.33$$

--- Gantt Chart ---



Comparison:	Pre-emptive	Non-pre-emptive
Avg-TAT	7.33	8.00
Avg-WT	3.33	4.00

∴ Pre-emptive is better & optimal

8/3
10/3

Week-3

Round Robin (Experiment with different quantum sizes for RR algorithm)

```
#include <stdio.h>
```

```
struct process {
    int pid; int at;
    int bt;
    int rt;
    int wt;
    int tot;
};
```

```
int main() {
    int n, quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    struct process p[n];
    for (int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("Enter burst time for process %d: ", p[i].pid);
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
```

```
        printf("Enter time quantum: ");
        scanf("%d", &quantum);
```

```
    int time = 0;
    int done;
```

```
int gc[100], gcTime[100], gcIndex=0;
```

```
while (1) {
    done = 1;
    for (int i=0; i<n; i++) {
        if (p[i].rt > 0) {
            done = 0;
            if (p[i].rt > quantum) {
                time += quantum;
                p[i].rt -= quantum;
                gc[gcIndex] = p[i].pid;
                gcTime[gcIndex++] = time;
            } else {
                time += p[i].rt;
                p[i].wt = time - p[i].bt;
                p[i].rt = 0;
                gc[gcIndex] = p[i].pid;
                gcTime[gcIndex++] = time;
```

```
            }
        }
        if (done == 1)
            break;
```

```
    }
    for (int i=0; i<n; i++) {
        p[i].tot = p[i].bt + p[i].wt;
```

```
    }
    printf("In Process |t Burst |t Waiting Time |t Turnaround Time |n");
```

```
    for (int i=0; i<n; i++) {
        printf("P %d |t %d |t %d |t %d |n",
            p[i].pid, p[i].bt, p[i].wt, p[i].tot);
```

Process	p1	p2	p3
Burst time	5	2	4
Waiting time	5	3	7
Turnaround time	10	5	11

Output:
 Enter number of process: 3
 Enter burst time for process 1: 5
 Enter burst time for process 2: 2
 Enter burst time for process 3: 4
 Enter time quantum: 3

return 0;

printf("\n");

printf("\n--p%d--\td", gcc[i], gcc[i]);
 for (int i = 0; i < gcc[i]; i++) {

printf("\n Gantt chart = \n");

printf("\n Average Waiting Time = %.2f", avgWT);
 printf("\n Average Turnaround Time = %.2f", avgTAT);

avgWT /= n;
 avgTAT /= n;

for (int i = 0; i < n; i++) {
 avgWT += p[i].wt;
 avgTAT += p[i].tat;

Process	p1	p2	p3
Burst time	4	5	8
Waiting time	4	10	15
Turnaround time	8	15	17

Gantt chart:
 Avg. Waiting Time = 6.00
 Avg. Turnaround Time = 11.33

Process	p1	p2	p3
AT	8	2	1
BT	4	5	7
WT	4	10	4
TAT	8	15	11
CT	16	17	12

Output:
 Enter number of processes: 3
 Enter arrival time of process 1: 8
 Enter burst time for process 1: 4
 Enter arrival time of process 2: 2
 Enter burst time for process 2: 5
 Enter arrival time for process 3: 1
 Enter burst time for process 3: 7
 Enter time quantum: 4

Process	p1	p2	p3
Burst time	4	5	8
Waiting time	4	10	15
Turnaround time	8	15	17

Gantt chart:
 Avg. Waiting Time = 5.00
 Avg. Turnaround Time = 8.67

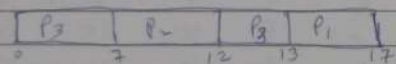
Enter number of processes = 3
 Enter arrival time for process 1: 8
 Enter burst time for process 1: 4
 Enter arrival time for process 2: 2
 Enter burst time for process 2: 5
 Enter arrival time for process 3: 1
 Enter burst time for process 3: 7
 Enter time quantum: 6

Process	AT	BT	WT	TAT	CT
P ₁	8	4	5	9	17
P ₂	2	5	5	10	12
P ₃	1	7	5	12	13

Avg. waiting Time = 5.00

Avg. ~~response~~ Turnaround Time = 10.33

Gantt chart:



Waiting Time and Turnaround Time is less when Time quantum is more.

17/3

Multilevel queue - week 6

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

typedef struct {
    int process_id, arrival_time, burst_time,
    remaining_time, completion_time,
    waiting_time, turnaround_time;
    char type[10];
} Process;
  
```

```

typedef struct {
    Process queue[MAX];
    int front, rear;
} Queue;
  
```

```

void initQueue (Queue *q) {
    q->front = q->rear = -1;
}

int isEmpty (Queue *q) {
    return q->front == -1;
}

void enqueue (Queue *q, Process p) {
    if (q->rear == MAX-1) return;
    if (q->front == -1) q->front = 0;
    q->queue[++q->rear] = p;
}
  
```

```

Process dequeueFCFS (Queue *q) {
    Process p = q->queue[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return p;
}

```

```

Process dequeueSTF (Queue *q) {
    int minIndex = q->front;
    for (int j = q->front; j <= q->rear; j++) {
        if (q->queue[j].remainingTime < q->queue[
            minIndex].remainingTime) {
            minIndex = j;
        }
    }
    Process p = q->queue[minIndex];
    for (int j = minIndex; j <= q->rear; j++) {
        q->queue[j] = q->queue[j+1];
    }
    q->rear--;
    if (q->rear < q->front) q->front = q->rear;
    return p;
}

```

```

int main() {
    int n;
    printf("Enter total number of processes: ");
    scanf("%d", &n);
    Process processList[MAX];
    for (int i = 0; i < n; i++) {
        printf("Enter process ID, arrival time, burst time, type (system/user): ");
        scanf("%d %d %d %c", &processList[i].id, &processList[i].arrivalTime, &processList[i].burstTime, &processList[i].type);
    }
}

```

```

processList[i].id, &processList[i].arrivalTime,
&processList[i].burstTime, processList[i].type);
processList[i].remainingTime = processList[i].burstTime;
}

```

```

3
queue(processList, n, sizeof(Process), compareTime);
int currentTime = 0, completed = 0, i = 0;
Process *currentProcess = NULL;
int timeline[MAX * 10]; timelineLen = 0;
while (completed < n) {
    while (i < n && processList[i].arrivalTime <= currentTime) {
        if (strcmp(processList[i].type, "system") == 0) {
            enqueue(&systemQueue, processList[i]);
        } else {
            enqueue(&userQueue, processList[i]);
        }
        i++;
    }
    if (currentProcess != NULL) {
        if (strcmp(currentProcess->type, "user") == 0 && !isEmpty(&systemQueue)) {
            enqueue(&systemQueue, currentProcess);
            currentProcess = NULL;
        }
    }
}

```

```

timeline[timelineLen++] = currentProcess->id;
currentProcess->remainingTime--;
currentTime++;

```

```

if (currentProcess->remainingTime == 0) {
    currentProcess->completionTime = currentTime;
}

```

$\text{current_process} \rightarrow \text{turnaround_time} = \text{current_process} \rightarrow \text{completion_time} - \text{current_process} \rightarrow \text{arrival_time}$
 $\text{current_process} \rightarrow \text{waiting_time} = \text{current_process} \rightarrow \text{turnaround_time} - \text{current_process} \rightarrow \text{burst_time}$

```

printf("\n Gantt Chart: \n");
for (int t=0; t<timeline.length; t++) {
    if (timeline[t] == -1) printf("idle");
    else printf("P%d", timeline[t]);
}

```

```

double avgWT=0, avgTAT=0;
printf("\n Process | AT | BT | CT | TAT | WT | RT | type\n");
for (int i=0; i<n; i++) {
    printf("P%d | %d | %d | %d | %d | %d | %d | %s\n", process_list[i].pid, at, bt, ct, tat, wt, rt, type);
    avgWT += process_list[i].waiting_time;
    avgTAT += process_list[i].turnaround_time;
}

```

```

printf("Average WT = %.2f \n", avgWT/n);
printf("Average TAT = %.2f \n", avgTAT/n);
return 0;

```

3

O/P Enter no. of Process : 3
 Enter AT, BT and type : 0 3 user
 Enter AT, BT and type : 1 4 system
 Enter AT, BT, and type : 3 5 system

Process	AT	BT	CT	TAT	WT	RT	type
P1	0	3	2	12	9	0	user
P2	1	4	5	4	0	0	system
P3	3	5	10	7	2	2	system

Gantt chart.

P1, P2, P3, P1, P2, P3

Avg WT = 3.67
 Avg TAT = 7.67

Queue's Summary:

System Queue : P2 P3
 user Queue : P1

of chunk

8/12

Week-5

Write a C program to simulate Real-time CPU Scheduling algorithm

- (i) Rate - Monotonic
- (ii) Earliest - Deadline First.

```
#include <stdio.h>
#define MAX 10
typedef struct {
    int id, bt, deadline, period;
    int ct, wt, tat;
    int remaining;
} process;
```

```
int gcd(int a, int b) {
    while(b) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int hyperperiod(process p[], int n) {
    int hp = p[0].period;
    for (int i = 1; i < n; i++)
        hp = lcm(hp, p[i].period);
    return hp;
}
```

```
void edf(process p[], int n) {
    process temp[MAX];
    for (int i = 0; i < n; i++) temp[i] = p[i];
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (temp[i].deadline > temp[j].deadline) {
                process t = temp[i];
                temp[i] = temp[j];
                temp[j] = t;
            }
}
```

```
int time = 0;
printf("m -- Earliest Deadline First -- \n");
printf("Gantt Chart: \n");
for (int i = 0; i < n; i++) {
    printf("P: %d \t", temp[i].id);
    time += temp[i].bt;
    temp[i].ct = time;
    temp[i].tat = temp[i].ct;
    temp[i].wt = temp[i].tat - temp[i].bt;
}
```

```
printf("\n");
time = 0;
```

```
for (int i = 0; i < n; i++) {
    time += temp[i].bt;
    printf(" %d \t", time);
}
```

```
printf("\n\n ID BT Deadline CT WT TAT \n");
for (int i = 0; i < n; i++) {
    printf("%d \t %d \t %d \t %d \t %d \t %d \n",
        temp[i].id, temp[i].bt, temp[i].deadline,
        temp[i].ct, temp[i].wt, temp[i].tat);
}
```

```

void rma (Process p[], int n) {
    Process temp[1000];
    for (int i=0; i<n; i++) temp[i] = p[i];
    for (int i=0; i<n; i++)
        for (int j=i+1; j<n; j++)
            if (temp[i].period > temp[j].period) {
                Process t = temp[i];
                temp[i] = temp[j];
                temp[j] = t;
            }
}

```

```

int time = 0;
printf("\n --- Rate monotonic Scheduling --- \n");
printf("Enter (n): ");
for (int i=0; i<n; i++) {
    printf("P%d: ", temp[i].id);
    time += temp[i].bt;
    temp[i].ct = time;
    temp[i].tat = temp[i].ct;
    temp[i].wt = temp[i].tat - temp[i].bt;
}

```

```

printf("\n 0");

```

```

time = 0;
for (int i=0; i<n; i++) {
    time += temp[i].bt;
    printf(" %d", time);
}

```

```

printf("\n In ID, BT, Period, CT, WT, TAT, ID");

```

```

for (int i=0; i<n; i++) {
    printf("%d %d %d %d %d %d %d",
        temp[i].id, temp[i].bt, temp[i].period,
        temp[i].ct, temp[i].wt, temp[i].tat,
        temp[i].id);
}

```

```

int main() {
    int n;
    Process p[1000];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        p[i].id = i;
        printf("In Process: %d: \n", i);
        printf("Enter time: ");
        scanf("%d", &p[i].bt);
        printf("Deadline (for EDF): ");
        scanf("%d", &p[i].deadline);
        printf("Period (for RMS): ");
        scanf("%d", &p[i].period);
    }
}

```

```

edit (p, n);
rma (p, n);
return 0;

```

Output:

```

Enter number of processes: 3
Enter process details:
Process 0:
Burst Time: 3
Deadline (for EDF): 7
Period (for RMS): 20

```

```

Process 1:
Burst Time: 2
Deadline (for EDF): 4

```

Period (for RMS) = 5

Process 2:

Burst Time = 2

Deadline (for EDF) = 8

Period (for RMS) = 10

=== Earliest Deadline First ===

CPU Utilization = 0.75

Schedulable (Utilization ≤ 1)

ID	BF	Deadline	CT	WT	TAT
0	3	7	5	2	5
1	2	4	2	0	2
2	2	8	7	5	7

=== Rate Monotonic Scheduling ===

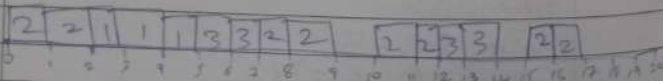
CPU Utilization = 0.75

RM Bound = 0.7798

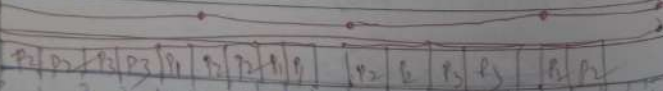
Schedulable (Utilization $\leq$ RM Bound)

ID	BF	Period	CT	WT	TAT
0	3	20	7	4	7
1	2	5	2	0	2
2	2	10	4	2	4

for EDF:



for RMS:



P1	3	20
P2	2	5
P3	2	10

$$\frac{3}{20} + \frac{2}{5} + \frac{2}{10}$$

8/5

3/2

Week-6

Q: Consider two processes A & B. Two semaphore variables S & T are used to synchronize the processes. S is initialized to 0 & T is initialized to 1. Process are scheduled in the following order. A B A B B A A B A. What will be printed?

Process A: while(1) { wait(S); print('A'); signal(T); }
 Process B: while(1) { wait(T); print('B'); signal(S); }

Process	S = 0	T = 1	Remark
A	0	1	Process A is blocked
B	1	0	i i
A	0	1	i i p p
B	1	0	i i p p i i
B	1	0	Process B is blocked
A	0	1	i i p p i i p p
A	0	1	Process A is blocked
B	1	0	i i p p i i p p i i
A	0	1	i i p p i i p p i i p p

Trace Table:

step	Action	Printed Output	in	out	buffer index used
1	P	Produced 0 at buffer[0]	1	0	0
2	C	Consumed 0 from buffer[0]	1	1	0
3	P	Produced 1 at buffer[1]	2	1	1
4	C	Consumed 1 from buffer[1]	2	2	1
5	P	Produced 2 at buffer[2]	0	2	2
6	P	Produced 3 at buffer[0]	1	2	0
7	C	Consumed 2 from buffer[2]	1	0	2
8	P	Produced 4 at buffer[1]	2	0	1
9	C	Consumed 3 from buffer[0]	2	1	0
10	P	Produced 5 at buffer[2]	0	1	2
11	P	Produced 6 at buffer[0]	1	1	0
12	C	Consumed 4 from buffer[1]	1	2	1

Dining Philosopher's problem

CS = [1, 1, 1, 1, 1]

P0 is thinking

P1 is thinking

P2 is thinking

P3 is thinking

P4 is thinking

P4 picked left fork 4 → CS = [1, 1, 1, 1, 0]

P1 picked left fork 1 → CS = [1, 0, 1, 1, 0]

P1 picked right fork 2 → CS = [1, 0, 0, 1, 0]

P1 is eating

P3 picked left fork 3 → CS = [1, 0, 0, 0, 0]

P0 picked left fork 0 → CS = [0, 0, 0, 0, 0]

P1 Put down forks 1 & 2 → CS = [0, 1, 1, 0, 0]

P1 is thinking

P2 picked left fork 2 → CS = [0, 1, 0, 0, 0]

P0 picked right fork 1 → CS = [0, 0, 0, 0, 0]

P0 is eating

P0 put down forks 0 & 1 → CS = [1, 1, 0, 0, 0]

P4 picked right fork 0 → CS = [0, 1, 0, 0, 0]

P4 is eating

P0 is thinking

22/4/25

Write a C program to simulate
a) Bankers Algorithm for the purpose of
deadlock avoidance

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], avail[m],
    need[n][m];
    printf("Enter Allocation matrix: \n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter maximum Demand Matrix: \n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available Matrix: \n");
    for (i=0; i<m; i++)
        scanf("%d", &avail[i]);
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    int work[m];
    for (i=0; i<m; i++) work[i] = avail[i];
    bool finish[n];
    for (i=0; i<n; i++) finish[i] = false;
```

```
int safeSeq[n], ind=0;
int safeSeqIn, ind=0;
for (k=0; k<n; k++) {
    for (i=0; i<m; i++)
        if (finish[i] == false) {
            int flag=0;
            for (j=0; j<m; j++)
                if (need[i][j] > work[j]) {
                    flag=1;
                    break;
                }
            if (flag==0) {
                for (int y=0; y<m; y++)
                    work[y] += alloc[i][y];
                finish[i] = true;
                safeSeq[ind++] = i;
            }
        }
    }
}
```

```
bool isSafe = true;
if (isSafe == true)
    for (i=0; i<n; i++)
        if (finish[i] == false) {
            isSafe = false;
            break;
        }
    if (isSafe)
        printf("\n System is in a safe state!\n");
        printf("Safe sequence is: ");
        for (i=0; i<n-1; i++) printf("%d -> ", safeSeq[i]);
        printf("%d\n", safeSeq[n-1]);
    }
}
```

```

else
    printf("System is in an UNSAFE state (Potential deadlock).");
}
return 0;
}

```

Output:

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

2 3 2 2

9 0 2

2 2 2

4 3 3

Enter available matrix:

3 2 2

System is in safe state.

Safe sequence is: P1 → P3 → P4 → P0 → P2

8/8/23

// Function for resource request Algorithm.
bool requestResources(int n, int m, int alloc[n][m], int need[n][m], int avail[m], int process, int request[]) {

for (int i=0; i<m; i++) {

if (request[i] > need[process][i]) {

printf("Error: Process has exceeded its maximum claim.\n");

return false;

for (int i=0; i<m; i++) {

if (request[i] > avail[i]) {

printf("Resources not available. Process must wait.\n");

return false

for (int i=0; i<m; i++) {

avail[i] -= request[i];

alloc[process][i] += request[i];

need[process][i] -= request[i];

int safeSeg[n];

if (isSafe(n, m, alloc, need, avail, safeSeg)) {

printf("Request can be granted, system remains in safe state.\n");

printf("Safe Sequence: ");

for (int i=0; i<n; i++)

printf("%d → ", safeSeg[i]);

printf("%d\n", safeSeg[n-1]);

return true;

}

```

else {
    for (int r=0; r<m; r++) {
        avail[r] += request[r];
        alloc[process][i] -= request[r];
        need[process][i] += request[r];
    }
    printf("request cannot be granted.\n");
    return false;
}

```

Output:

Safe sequence: P1 → P3 → P4 → P0 → P2

Enter process number making request: 1
Enter request vector:

1 0 2

Request can be granted. System is in safe state.

Safe sequence: P1 → P3 → P4 → P0 → P2

Req.

b) Deadlock Detection

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[n][m], request[n][m], avail[m];
    bool finish; int deadlock=0, count=0;
    printf("\n Enter Allocation Matrix:");
    for (int i=0; i<n; i++) {
        finish[i] = false;
        for (int j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\n Enter Request Matrix:");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("\n Enter Available resources:");
    for (int i=0; i<m; i++) {
        scanf("%d", &avail[i]);
    }
    for (int k=0; k<n; k++) {
        for (int i=0; i<m; i++) {
            if (!finish[i]) {
                bool canBeDone = true;
                for (int j=0; j<m; j++) {
                    if (request[i][j] > avail[j]) {
                        canBeDone = false;
                    }
                }
                if (canBeDone) {
                    // ... (rest of the code)
                }
            }
        }
    }
}

```

```

if (canBeDone) {
    for (int j=0; j<m; j++)
        avail[j] += alloc[i][j];
    finish[i] = true;
}
}
}
for (int i=0; i<n; i++) {
    if (!finish[i])
        deadlockedCount++;
}
if (count > 0) {
    printf("The system is in DEADLOCK. Deadlocked processes: ");
    for (int i=0; i<count; i++)
        printf("P%d", deadlocked[i]);
    printf("\n");
} else {
    printf("No Deadlock Detected!\n");
}
return 0;
}

```

Output:

Enter number of processes: 4

Enter number of resource types: 3

Enter Allocation Matrix:

1 0 2

2 1 1

1 0 3

Enter Request Matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter Available Matrix:

0 0 0

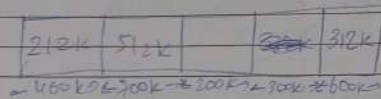
System is in Deadlock
Deadlocked processes: P3

Ans
5/6/20

Contiguous Memory Allocation.

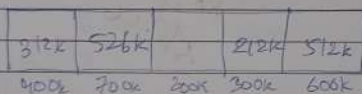
Given a memory partition 400k, 700k, 200k, 300k, 600k sequentially. how would each of First Fit, best fit, worst fit algorithms work do place process of 212k, 512k, 312k, 52k in order.

First Fit

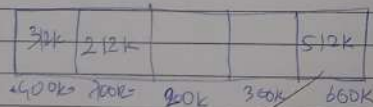


52k cannot be allocated with memory.

Best fit



Worst fit



52k cannot be allocated with memory.

C-code:

```
#include <stdio.h>
void simulate(int b[], int m, int p[], int n,
char* name, int mode) {
    int alloc[20], blocks[20];
    for (int i=0; i<m; i++) blocks[i]=b[i];
    for (int i=0; i<n; i++) alloc[i]=-1;
```

```
for (int i=0; i<m; i++) {
    int idx=-1;
    for (int j=0; j<n; j++) {
        if (blocks[j] >= p[i]) {
            if (mode==0) {idx=j; break;}
            if (mode==1 && (idx==-1)) {
                blocks[j] < blocks[idx]; idx=j;
            }
            if (mode==2 && (idx==-1)) {
                blocks[j] > blocks[idx]; idx=j;
            }
        }
    }
    if (idx!=-1) alloc[i]=idx;
    blocks[idx]-=p[i];
}
```

```
printf("\n --- %s --- \n\n block size & block name");
for (int i=0; i<n; i++) {
    printf("%d\t %d\t %s\n", i+1, p[i],
    alloc[i] != -1 ? (char*)alloc[i] : "None");
}
```

int main()

```
int m, n, b[20], p[20];
printf("Enter # blocks & sizes:");
scanf("%d", &m);
for (int i=0; i<m; i++) scanf("%d", &b[i]);
printf("Enter # process & sizes:");
scanf("%d", &n);
for (int i=0; i<n; i++) scanf("%d", &p[i]);
```

```

simulate(b, m, p, n, "FIRST FIT", 0);
simulate(b, m, p, n, "BEST FIT", 1);
simulate(b, m, p, n, "WORST FIT", 2);
return 0;
}

```

Output:

Enter # Blocks & Sizes = 5
400 700 200 300 600

Enter # Process & Sizes = 4

212

512

312

526

--- FIRST FIT ---

Proc	Size	Block
1	212	1
2	512	2
3	312	3
4	526	None

--- BEST FIT ---

Proc	Size	Block
1	212	4
2	512	5
3	312	1
4	526	2

--- WORST FIT ---

Proc	Size	Block
1	212	2
2	512	5
3	312	1
4	526	None

Page replacement Algorithm

Write a C program to implement page replacement algorithm using
(i) FIFO (ii) LRU (iii) Optimal.

C-code

```

#include <stdio.h>
#include <stdbool.h>

```

```

void simulate(int pages[], int n, int nf,
char *name, int type) {
    int frames[20], time[20] = {0}, fault = 0;
    for (int i = 0; i < nf; i++)
        frames[i] = -1;
    printf("\n --- %s Simulation ---\n",
name);
    for (int i = 0; i < n; i++) {
        bool hit = false;
        for (j = 0; j < nf; j++) {
            if (frames[j] == pages[i]) {
                hit = true;
                break;
            }
        }
        if (hit) {
            printf("Page %d: HIT\n",
pages[i]);
        } else {
            int idx = 0;
            for (int j = 0; j < nf; j++) {
                if (frames[j] == -1) {
                    idx = j;
                    break;
                }
            }

```

```

if (type == 0 && j == faults % nf) {
    idx = j;
} else if (type == 1 && time[j] < time[idx]) {
    idx = j;
} else if (type == 2) {
    int k, for = i; current_idx = for;
    for (int frame = j; frame < nf; frame++) {
        for (k = i + 1; k < n; k++) {
            if (frames[frame] == pages[k])
                break;
        }
        if (k == n) {
            idx = frame - j;
            break;
        }
        if (k > current_idx - for) {
            current_idx = for = k;
            idx = frame - j;
        }
    }
}
}
}

```

```

frames[idx] = pages[i];
faults++;

```

```

printf("Page %d, MISS (Fault) \n", pages[i]);

```

```

for (int j = 0; j < nf; j++)
    if (frames[j] == pages[i]) time[j] = i;

```

```

printf("Total page faults (%.5f): %d \n", n, faults);

```

```

int main() {
    int n, nf, pages[50];
    printf("Enter number of frames: ");
    scanf("%d", &nf);
    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter the page reference string separated by spaces: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    simulate(pages, n, nf, "FIFO", 0);
    simulate(pages, n, nf, "LRU", 1);
    simulate(pages, n, nf, "Optimal", 2);
    return 0;
}

```

Output:

Enter number of frames: 3

Enter number of pages (length of string): 20

Enter string with spaces between letters:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 3 0 1

--- FIFO ---

Page 7: Miss (Fault)

Page 0: Miss (Fault)

Page 1: Miss (Fault)

Page 2: Miss (Fault)

Page 0: Hit

Page 3: Miss (Fault)

Page 0: Miss (Fault)

Page 4: Miss (Fault)

Page 2: Miss (Fault)

Page 3: Miss (Fault)

Page 0: Miss (Fault)

Page 3: Hit

Page 2: Hit

Page 1: Miss (Fault)

Page 2: Miss (Fault)

Page 0: Hit

Page 1: Hit

Page 7: Miss (Fault)

Page 0: Miss (Fault)

Page 1: Miss (Fault)

Total Page Faults (FIFO) = 15

--- LRU ---

Page 7: Miss (Fault)

Page 0: Miss (Fault)

Page 1: Miss (Fault)

Page 2: Miss (Fault)

Page 0: Hit

Page 3: Miss (Fault)

Page 0: Hit

Page 4: Miss (Fault)

Page 2: ~~Miss~~ Miss (Fault)

Page 3: ~~Hit~~ Miss (Fault)

Page 0: Miss (Fault)

Page 3: Hit

Page 2: Hit

Page 1: Miss (Fault)

Page 2: Hit

Page 0: ~~Miss~~ Miss (Fault)

Page 1: Hit

Page 7: Miss (Fault)

Page 0: Hit

Page 1: Hit

Total Page Faults (LRU) = 12

----- Optimal -----

Page 7: Miss (Fault)

Page 0: Miss (Fault)

Page 1: Miss (Fault)

Page 2: Miss (Fault)

Page 0: Hit

Page 3: Miss (Fault)

Page 0: Hit

Page 4: Miss (Fault)

Page 2: Hit

Page 3: Hit

Page 0: Miss (Fault)

Page 3: Hit

Page 2: Hit

Page 1: Miss (Fault)

Page 2: Hit

Page 0: Hit

Page 1: Hit

Page 7: Miss (Fault)

Page 0: Hit

Page 1: Hit

Total Page Faults (Optimal) = 9

 21/6